Gym Membership Management System

A Java back-office for a gym, built around two design patterns.

SEN3006 | Software Architecture | June 2026

Elif Yıldırım ·

presenter, builder pattern, lifecycle enum

Teammate 2 ·

presenter, observer pattern, demo driver

AGENDA

What we will cover today

#	Topic	Who
1	The gym problem in plain English	E
2	What we actually built	E
3	Architecture, one diagram	T2
4	Builder, and why we used it	T2
5	Observer, and why we used it	E
6	Lifecycle as a state machine	T2
7	Live demo	T2 drives, E narrates
8	Extending the system	E
9	SOLID, fast	E
10	Wrap up and questions	Both

1. The problem

Two things every gym back-office has to do

Membership plans have too many fields

A plan is a name, a duration, a monthly fee, an access tier, a list of included classes, a guest-pass quota, a freeze allowance, and a personal-trainer flag. That is eight fields, and only the name is really required.

```
// The obvious constructor is unreadable:
new MembershipPlan("Gold", 12, 49.99, PREMIUM,
    Set.of("yoga", "spin"), 2, 30, true);
```

If we add a ninth field next month, every place that creates a plan has to change. Telescoping constructors get worse with each one we add, not better.

> **Creational problem.** This is where Builder earns its place.

Notifications fan out per member

Each member picks their own mix of email, SMS, and push. A payment-due event goes to one member's channels. A promotion goes to everyone's channels. A class cancellation can do either, depending on whether one person booked it or many did.

```
// Without a pattern, the publishing code grows branches:
if (member.wantsEmail())  emailService.send(...);
if (member.wantsSms())    smsService.send(...);
if (member.wantsPush())   pushService.send(...);
```

Repeat that block in every event method. Add WhatsApp later, and we have to edit every call site.

> **Behavioral problem.** This is where Observer earns its place.

2. What we built

One Java engine, three ways to drive it

The application

Engine. A single `Gym` aggregator that owns members and plans, and publishes events to attached notifiers.

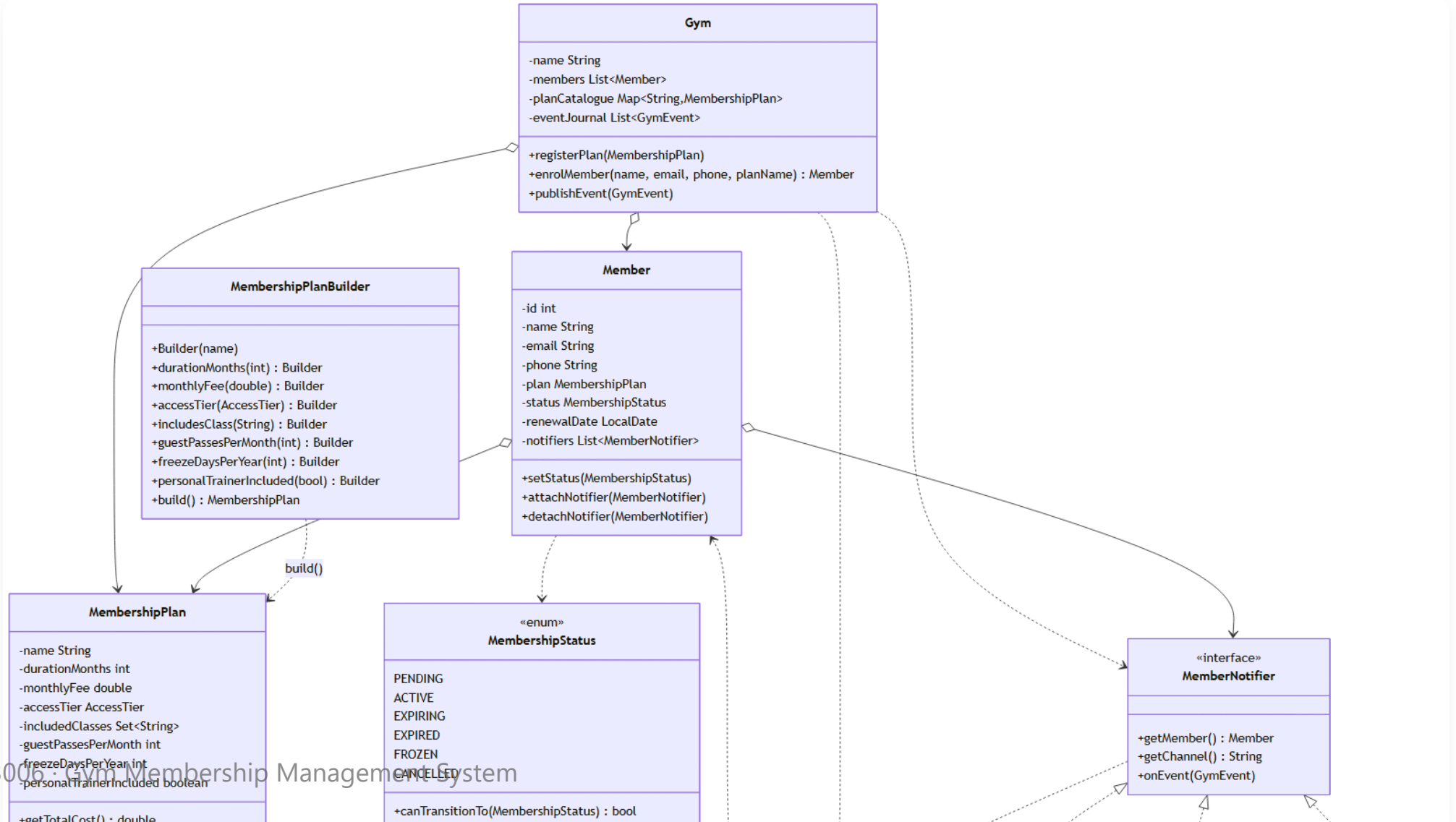
How we run it (terminal, as the brief asks)

1. `java -cp bin Main` runs six self-checking test sections from the command line.
2. `java -cp bin GymManagementApp` opens an interactive console menu in the same terminal.

Numbers

- 19 Java files
- 8 plan fields
- 4 event types
- 3 notifier channels
- 6 lifecycle states

The class diagram



4. Builder

Why we picked it for `MembershipPlan`

A plan should read like a sentence, not a riddle

A `MembershipPlan` has eight fields, and most of them are optional. With an eight-argument constructor, the call site is a row of numbers and flags that nobody can decode without opening another file. With a Builder, the same call reads like the receptionist describing the plan out loud: "Gold Annual, twelve months, 49.99 a month, premium tier, includes yoga and spin, freeze allowance thirty days, personal trainer on, build it."

One more thing the chain gives us. `build()` is the single place validation runs. If anyone passes `monthlyFee(-1)`, it throws there, before any half-formed plan ever escapes into the system.

```
MembershipPlan gold = new MembershipPlan.Builder("Gold Annual")
    .durationMonths(12)
    .monthlyFee(49.99)
    .accessTier(AccessTier.PREMIUM)
    .includesClass("yoga")
    .includesClass("spin")
    .freezeDaysPerYear(30)
    .personalTrainerIncluded(true)
    .build();
```

Why a Builder, not a POJO with setters

A gym plan is a contract. If a member signs up on Gold Annual at 49.99 a month, that price must not change underneath them six months later. Setters would allow exactly that, because anyone holding a reference to the plan could rewrite its fields at any moment. Builder seals the plan after `build()` returns, so the contract stays fixed for the lifetime of the membership. That is the deciding line for us.

Concern	Setters POJO	Builder
8 optional fields	OK	OK
Validation lives in one place	Hard, scattered	Yes, inside `build()`
Plan stays immutable after creation	No, setters defeat it	Yes, no public setters
Partial or half-built objects can leak	Yes	No
Pricing can drift after signup	Yes	No

5. Observer

Why we picked it for notifications

One line says "this happened", every channel handles itself

When a member's payment is due, the gym needs to tell them through every channel they signed up for. Without Observer, every code path that fires an event would have to remember every channel: email if they want email, SMS if they want SMS, push if they want push, repeated everywhere a notification can fire. With Observer, the business code says "this happened" once, and each channel the member has attached handles itself.

```
// One line in business logic:
gym.publishPaymentDue(memberId, dueDate, 49.99);
```

Internally, that walks the target member's notifier list. If the member has `EmailMemberNotifier` and `SmsMemberNotifier` attached, both fire. Each one formats the message in its own voice. Email gets a full paragraph, SMS gets one line, push gets a 40-character headline. `Gym` never knows what the message looks like on any channel.

Different events carry different things, so we typed them

Different events carry different information. A payment-due event needs an amount and a date. A class cancellation needs a class name. A promotion needs a discount percent. If we passed all of these as plain strings, the `Gym` class would have to format every message itself, which mixes domain code with presentation. Typed events let each notifier format the message in its own voice, on its own channel, without the `Gym` knowing anything about wording.

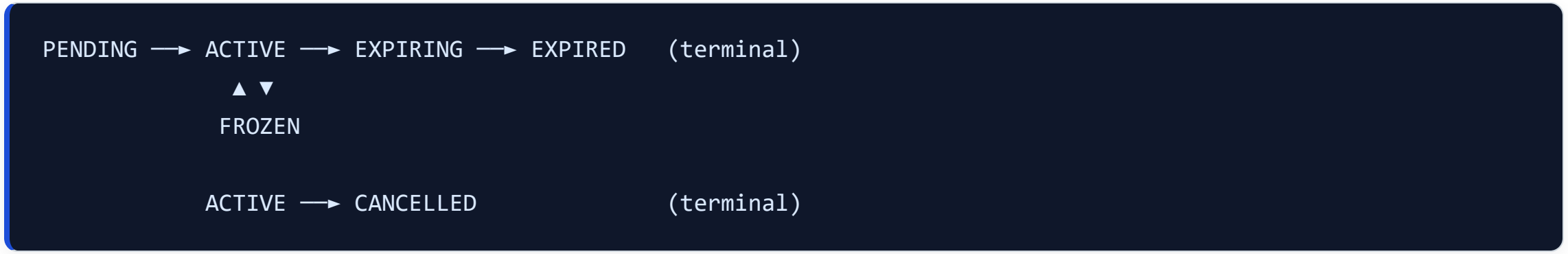
The other choice worth flagging: notifiers attach to the member, not to the `Gym`. A member who only wants SMS should not show up in another member's channel list. Targeted events walk one member's notifiers, broadcasts walk every member's in turn.

Event	Payload	Targeting
`PaymentDueEvent`	amount, due date	one member
`RenewalReminderEvent`	renewal date	one member
`ClassCancelledEvent`	class name, date	one or all
`PromotionEvent`	discount percent	always all

6. Lifecycle

A third pattern, almost for free

`MembershipStatus` is its own state machine



Each enum constant declares its own `allowedTransitions()`. `Member.setStatus(...)` checks that set and throws `IllegalArgumentException` on any illegal move.

> We get the State pattern out of a single enum file. No extra classes, no scattered conditional logic, no boolean flags to forget.

7. Live demo

About three minutes, on the laptop

DEMO · T2 drives, E narrates

What we will show (all in the terminal)

1. **Scripted self-checks.** `java -cp bin Main`. Six test sections run, each printing one `[PASS]`. Pause on Section 6, where one published event prints three differently formatted messages, one per channel.
2. **Interactive console.** `java -cp bin GymManagementApp`. Open the menu, register a plan, enrol a member, attach Email and SMS.
3. **Trigger Builder validation.** Choose "create plan", set `monthlyFee` to `-1`. The engine throws, the console prints the exact reason.
4. **Trigger Observer fan-out.** Publish a payment-due event. Two notifier lines print to the console for the one member, one for Email, one for SMS.
5. **Trigger an illegal transition.** Try moving a `CANCELLED` member back to `ACTIVE`. The engine refuses with the exact reason.

8. Extending it

What changes if you add a channel, event, or field

Three "what if" cases

Add a WhatsApp channel

```
class WhatsAppMemberNotifier
    implements MemberNotifier { ... }

member.attachNotifier(
    new WhatsAppMemberNotifier(...));
```

Zero edits to Gym, to events, or to existing notifiers.

Add a `MemberBirthdayEvent`

```
class MemberBirthdayEvent
    extends GymEvent { ... }

gym.publishEvent(
    new MemberBirthdayEvent(member));
```

Zero edits to existing events.

Add `lockerIncluded` to plans

```
// MembershipPlan: 1 field + 1 accessor
// MembershipPlan.Builder: 1 setter
```

The Builder absorbs the change in one place. No other class touches it.

Why this matters

- Open/Closed in concrete file counts, not theory.
- New features without touching tested code.
- Test 5 in `Main.java` proves this at runtime by installing a brand-new anonymous notifier while the program is already running.

How the design lines up with SOLID

- **S, Single Responsibility.** The `Gym` coordinates only. Builders build, notifiers render, events carry data.
- **O, Open/Closed.** New channel, new event, new plan field. None of them edit existing code.
- **L, Liskov.** Every notifier and event subclass can stand in for its abstraction without surprises.
- **I, Interface Segregation.** `MemberNotifier` has three short methods, and every implementation uses all three.
- **D, Dependency Inversion.** The `Gym` references only abstractions. There is no `new EmailMemberNotifier(...)` anywhere inside `Gym`.

9. Wrapping up

What worked, and what we would do next

Honest reflection

What worked

- Builder kept the call sites readable and the plans immutable.
- Observer with typed events kept the `Gym` free of presentation logic.
- The lifecycle enum gave us a small state machine without writing a new class.
- The scripted `Main` doubled as our test harness without bringing in JUnit.

What we would do next

- Persistence. Right now everything lives in memory and dies with the JVM.
- Async notifier dispatch. The current fan-out is synchronous, so a slow channel blocks the rest.
- Bulk batching for promotions, since today every member triggers a separate walk.
- A small REST layer on top, reusing the same engine.

Questions?

Thank you for listening.

github.com/hoop-ai/gym-membership-management-system

Elif Yıldırım Teammate 2